

enc_ramdisk — AES-XTS Encrypted Volatile RAM Block Device

A loadable kernel module (`enc_ramdisk.ko`) that registers a block device (`/dev/enc_ramdisk0`) backed entirely by `vmalloc` 'd kernel memory. Every sector is encrypted with AES-256 in XTS mode via the Linux Kernel Crypto API before being written to the backing buffer, and decrypted on read. The encryption key is generated with `get_random_bytes()` at module load time, lives only in kernel memory, is never exposed to user space, and is wiped with `memzero_explicit()` on unload. Data does not survive `rmmod` /reboot — this is a RAM disk, not persistent storage.

"Hardware-enforced" here refers to the Kernel Crypto API's implementation selection: on any x86_64 CPU with AES-NI, the API will transparently pick

the hardware-accelerated `xts-aes-aesni` implementation over the generic C fallback — you get hardware-accelerated encryption for free, with no driver-side changes. It does not use a TPM, SGX, or another secure enclave; treat the name as marketing shorthand for "runs on the CPU's AES instruction set."

Architecture

- Block layer: `blk-mq`, single hardware queue, `BLK_MQ_F_BLOCKING` set because `queue_rq` sleeps while waiting on the crypto operation.
- Backing store: one contiguous `vzalloc()` buffer sized in module init.
- Crypto: `xts(aes)`, 64-byte key (AES-256-XTS: two 32-byte AES-256 keys concatenated, per the XTS spec).
- IV: sector number as a little-endian 64-bit integer padded to 16 bytes (the same `plain64` convention `dm-crypt` uses for XTS), computed fresh per sector so no external IV storage is needed.
- I/O path: `queue_rq` walks the request's `bio_vecs` sector-by-sector,

`kmap_local_page()` s each page, and encrypts (on `WRITE`) or decrypts (on `READ`) directly between the request's pages and the backing buffer.

- Verification hook: a debugfs file at `/sys/kernel/debug/enc_ramdisk/raw_ciphertext` exposes the raw backing buffer read-only (root only), so you can prove data at rest is actually ciphertext – reading through `/dev/enc_ramdisk0` always shows you plaintext, by design, since the driver decrypts transparently.

Kernel version target

Written against the `blk-mq` + `blk_mq_alloc_disk()` API as it existed from Linux 5.19 through roughly 6.8 (e.g. Ubuntu 22.04/24.04 HWE kernels, Debian 12). Block-layer internals move fast:

- Linux 6.9+ reworked queue limits; `blk_mq_alloc_disk()` gained a `struct queue_limits *` parameter on some later kernels, and several of the `blk_queue_*` setter calls used here were folded into that struct.

- Pre-5.19 kernels don't have `blk_mq_alloc_disk()` at all; you'd need the older `blk_mq_init_queue()` + `alloc_disk()` pairing.
- `GENHD_FL_NO_PART` has been renamed at least once historically (`GENHD_FL_NO_PART_SCAN` on older trees).

If `make` fails on your kernel, these are the first things to check against your kernel's `include/linux/blk-mq.h` and `blkdev.h`.

No kernel headers were available in the sandbox this was written in, so this could not be test-compiled — build and test it yourself before relying on it.

Build

```
sudo apt install build-essential linux-headers-$(uname -r) # Debian/Ubuntu
make
```

Load and test

Test this in a disposable VM, not your main machine.

A bug in a

block driver (or in this one) can hang or panic the kernel it's loaded into.

```
sudo insmod enc_ramdisk.ko erd_size_mb=64
dmesg | tail # confirm
it loaded, note the major number
lsblk | grep enc_ramdisk

sudo mkfs.ext4 /dev/enc_ramdisk0
sudo mount /dev/enc_ramdisk0 /mnt
echo "hello from plaintext-land" | sudo tee
/mnt/test.txt
sudo umount /mnt
```

Verify encryption at rest:

```
sudo dd if=/dev/enc_ramdisk0 bs=4K count=4
2>/dev/null | strings # via the driver:
readable
sudo cat
/sys/kernel/debug/enc_ramdisk/raw_ciphertext | head -c 4096 | xxd | head # raw
backing store: should look like noise
```

Unload:

```
sudo rmmod enc_ramdisk
```

Module parameters

- `erd_size_mb` (read-only sysfs, set at load time):
RAM disk size in
megabytes. Default 64.

Known limitations (by design, for scope)

- No quiescing of in-flight I/O on unload —
unmount/idle the device
before `rmmod`.
- No partition table support (`GENHD_FL_NO_PART`) —
it's a flat block
device, not a disk you partition.
- Same sector + same plaintext always produces
the same ciphertext
(standard XTS-per-sector behavior, same tradeoff
`dm-crypt` makes).
- Key is not recoverable or settable from user space
— losing the module
(or the machine's power) loses the data. That's
the point of a RAM
disk, but worth stating plainly.